Assignment - 5

Problem Statement:
Department of Computer Engineering has student's club named 'Pinnacle Club'. Students of second, third and final year of department can be granted membership on request. Similarly, one may cancel the membership of club. Write a program to maintain club member's information using singly linked list, store student PRN and Name. Write functions to: a) Add and delete the members as well as president or even secretary b) Compute total number of members of club c) Display members d) Sorting of the linked list e) Reversing using the pointers.

Objective:
1) To understand the concept of a singly linked list and dynamic memory allocation using the pointers in C.
2) To perform insertion, deletion, display, count nodes, sort, reverse operation on a singly linked list.
3) To develop a menu driven application for workshop enrollment management.

Implementation:
- Platform
  - 64 bit Open source Linux or its derivatives.
  - Open Source C Programming tool like gcc / Eclipse Editor.

- Test Conditions:-

Test Case	I / O	Expected output
TC1	Add the President	President PRN and Name are stored successfully as the first node
TC2	Add the Secretary	Secretary's PRN and Name are stored successfully as the first node

TC3	Add a regular club member	Student record is added successfully to the linked list.
TC4	Add multiple members	All student records are stored in the linked list in the specified order.
TC5	Display an empty club list	Message "No students registered
TC6	Display all club members	All student records including the President and Secretary are displayed correctly.
TC7	Delete an existing regular member	The selected student's record is deleted successfully
TC8	Delete the President or Secretary	The selected special member is removed successfully and the list links/head are updated correctly.
TC9	Delete a non-existing PRN	Message "Student not found"
TC10	Count club members	The correct total number of registered club members is displayed.
~~TC11~~	~~Sort the list by PRN~~	
TC11	Sort the list by PRN	Student records are displayed in ascending order of PRN.
TC12	Merge two linked lists	The records from both list are combined into one linked list in the specified order.
TC13	Reverse the linked list using three pointers	The order of student records is reverse correctly.
TC14	Delete all members and display	The list becomes empty and an appropriate message such as "No students registered", is displayed

PSUEDO code

Insertion

Algorithm InsertNode(head, prn, name, positionType):
  Create new node 'newNode'
  newNode.prn = prn
  newNode.name = name
  newNode.next = null

```
If positionType == "President" (Beginning):
    newNode.next = head
    head = newNode

Else If positionType == "Secretary" (End):
    If head is Null:
        head = newNode
    Else:
        temp = head
        While temp.next is not Null:
            temp = temp.next
        temp.next = newNode

Else (Regular Member):
    // Insert in middle or at specified position
```

B) Deletion

```
Algorithm DeleteNode(Head, targetPRN):
If head is Null:
    Display "List is empty"
    Return
&
If head.prn == targetPRN:
    temp = head
    head = head.next
    Free(temp)
    Return

curr = head
While curr is not Null and curr.next.prn != targetPRN:
    curr = curr.next
If curr.next is Null:
    Display "Student not found"
```

Else:
nodeToDelete = curr.next
curr.next = curr.next.next
Free (nodeToDelete)

C Display Members

Algorithm Display(head):
If head is Null:
Display "No student's registered."
Return
curr = head
While curr is not Null:
Print curr.prn, curr.name
curr = curr.next

D Count Nodes
Algorithm CountNodes(head):
count = 0
curr = head
While curr is not Null:
count = count + 1
curr = curr.next
Return count

E Sorted List (Bubble Sort by PRN)

Algorithm SortList(head):
If head is Null or head.next is Null:
Return
swapped = true
while swapped:
swapped = false

```
curr = head
while curr.next is not Null:
    if curr.prn > curr.next.prn:
        Swap(curr.prn, curr.next.prn)
        swap(curr.name, curr.next.name)
        swapped = True
    curr = curr.next
```

F) Reverse Linked List

```
Algorithm Reverse List (head):
    prev = Null
    curr = head
    nextPtr = Null
    while curr is not Null:
        next Ptr = curr.next
        curr.next = prev
        prev = curr
        curr = nextPtr
    hed = prev
    Return head
```

Time Complexity:

- Insert at Begining (Presidn): O(1)
- Insert at End (secretary), Delete, Search, Display, Count, Reverse: O(n)
- Sorting $O(n^2)$ eg. Bubble Sort)

Conclusion

1

FAQ

1) Why linked list / Node info?
Ans Allows dynamic size changes : store student PRN and Name

2) Singly linked list : Linear structure where each node points to the next, permitting one way traversal.

3) Node info: ~~Student PRN~~ and Name.
Ans PRN and Name

4) Dynamic memory Allocation
Ans Allocating memory manually during runtime using heap storage.

5) Release memory!
Ans free() function

6) Insert at Beginning TC:
Ans O(1)

7) Search TC:
Ans O(n)

8) Reversal method:
Ans Iteratively updating each node's pointer using prev, curr and next pointers.

9) Sorting techniques : Bubble Sort, Insertion, or Merge Sort.